

Structuring Machine Learning Projects

DL Specialization, Course 3 — detailed notes (Weeks 1–2)

ML strategy: metrics · human-level performance · error analysis · mismatched data · transfer / multi-task / end-to-end

This course has no math or code — it’s about *decisions*: given many ways to improve a model, which one actually moves the needle? Andrew’s framing: “I’ve seen teams waste months going in the wrong direction. These strategies save that time.”

1. Week 1a — Introduction to ML Strategy

1.1 Why ML strategy

You always have many ideas to improve a system: collect more data, train longer, try Adam, bigger/smaller network, dropout, L2, change architecture... Picking wrong wastes months. ML strategy is how to *analyze* which direction is worth it.

1.2 Orthogonalization

Tune one thing at a time, where each “knob” affects one outcome — like separate TV dials for size vs. position (vs. one knob that changes both, which is unusable). The **chain of assumptions**:

1. Fit the **training set** well (near human level). *Knobs*: bigger network, better optimizer (Adam).
2. Fit the **dev set** well. *Knobs*: regularization, bigger training set.
3. Fit the **test set** well. *Knobs*: bigger dev set.
4. Perform well in the **real world**. *Knobs*: change the dev set or the cost function.

Andrew avoids **early stopping** as a knob because it’s *not* orthogonal — it affects training-set fit *and* dev-set overfitting at once, so it’s harder to reason about.

2. Week 1b — Setting up your goal

2.1 Single number evaluation metric

Set *one* number before starting, so you can quickly compare models. Precision and recall alone don’t rank classifiers (one may win on each) — combine them with the **F1 score**:

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = \frac{2}{\frac{1}{P} + \frac{1}{R}} = 2 \frac{PR}{P + R} \text{ (harmonic mean).}$$

One number $\Rightarrow$ faster iteration. (Also: average performance across markets/classes into one figure.)

2.2 Satisficing and optimizing metrics

When you can’t reduce to one number, pick **one optimizing** metric and set the rest as **satisficing** (must-clear thresholds):

maximize accuracy subject to runtime < 100 ms.

General rule: 1 optimizing metric, $N - 1$ satisficing ones (e.g. wake-word: maximize accuracy, subject

to ≤ 1 false trigger per 24h).

2.3 Train / Dev / Test distributions

Dev and test sets **must come from the same distribution** — and should reflect the data you expect in the future and care about.

Choosing the dev set + metric *defines the target you aim at*. Don't set the target on one distribution then get graded on another (e.g. dev on data from some regions, test on others $\Rightarrow$ months wasted aiming wrong).

2.4 Size of dev and test sets

Old rule 60/20/20 was fine for $\sim 10^4$ examples. For 10^6+ , use e.g. 98/1/1. The test set only needs to be big enough for a *confident* estimate of final performance — not a fixed 30%.

2.5 When to change dev/test sets and metrics

Change them if doing well on your metric no longer means doing well on your application. Example: classifier A has 3% error but lets through offensive images; B has 5% but is clean — users prefer B, so the metric is wrong. Fix by *weighting* the errors:

$$\text{Error} = \frac{1}{\sum_i w^{(i)}} \sum_i w^{(i)} \mathbb{I}\{\hat{y}^{(i)} \neq y^{(i)}\}, \quad w^{(i)} = \begin{cases} 1 & \text{ok} \\ 10 & \text{offensive} \end{cases}.$$

This is orthogonalization again: (1) *define* a metric that captures what you want (place the target); (2) separately, do well on it (aim at the target). If metric + dev/test don't match the real task, change them.

3. Week 1c — Comparing to Human-level Performance

3.1 Why human-level performance

(1) Deep learning made ML competitive with humans on many tasks, so the comparison is now meaningful; (2) the workflow is more efficient when humans can also do the task (you can get labels, do error analysis, and estimate the ceiling). Progress is fast until you reach human level, then slows — because you approach **Bayes optimal error** (the theoretical best), and there's little gap between human and Bayes error on perception tasks.

3.2 Avoidable bias

Use human-level error as a *proxy* for Bayes error:

$$\begin{aligned} \text{Avoidable bias} &= \text{Training error} - \text{Human (Bayes) error}, \\ \text{Variance} &= \text{Dev error} - \text{Training error}. \end{aligned}$$

Same train/dev numbers, different action depending on the human baseline:

	Human 1%	Human 7.5%
Train error	8%	8%
Dev error	10%	10%
Focus on	bias (gap 7%)	variance (gap 2%)

You can't do better than Bayes error unless you overfit — so “avoidable” bias is the part worth chasing.

3.3 Understanding human-level performance

“Human level” should be chosen as the *best* achievable proxy for Bayes error. E.g. for medical images: a team of expert doctors < one expert < an ordinary doctor — use the *team of experts* figure as your Bayes-error estimate. Which gap (bias vs. variance) is bigger tells you what to work on.

3.4 Surpassing human-level performance

Once you *beat* the human baseline, you lose your Bayes-error estimate, so bias/variance analysis gets murky — progress slows. ML already surpasses humans on many *structured-data* tasks (online advertising, product recommendation, loan approval, logistics) — these aren’t natural perception. Beating humans on *perception* (vision, speech) is harder but increasingly done.

3.5 Improving your model performance (summary)

Two fundamental assumptions of supervised learning: (1) you can fit the training set well (low avoidable bias); (2) it generalizes to dev/test (low variance).

- **Reduce avoidable bias** (train – human): bigger model, train longer, better optimizer (momentum/RMSprop/Adam), better architecture/hyperparameters.
- **Reduce variance** (dev – train): more data, regularization (L2, dropout, augmentation), better architecture/hyperparameters.

4. Week 2a — Error Analysis

4.1 Carrying out error analysis

Before spending weeks on a fix, *manually* examine mistakes to estimate the payoff.

Grab ~100 mislabeled dev examples, count causes. If only 5% are (say) dogs, fixing dogs caps your gain at 10% → 9.5% (the **ceiling**) — not worth it. If 50% are dogs, fixing them could reach 10% → 5% — do it.

Evaluate multiple ideas in parallel with a spreadsheet: one row per image, columns for each error category (dog, great cats, blurry, filters...), tally the percentages, and work on the biggest bucket. A few hours of counting saves months of misdirected effort.

4.2 Cleaning up incorrectly labeled data

DL is robust to *random* label errors in the *training* set (less so to systematic ones) — often not worth fixing. For dev/test, add a “mislabeled” column in your error analysis:

If dev error is 10% and mislabeled-label errors account for 0.6%, the other 9.4% dominates — fix the model, not the labels. Fix labels when they’re a large share of remaining error.

Guidelines: apply the same fix to dev *and* test (keep them same-distribution); consider examples the model got *right* too (not just wrong); train and dev/test may now differ slightly in distribution — that’s tolerable.

4.3 Build your first system quickly, then iterate

Set up dev/test + metric → build a quick-and-dirty system → use bias/variance + error analysis to prioritize next steps. Iterate.

Don’t over-engineer at the start; the first system tells you where the real problems are.

5. Week 2b — Mismatched Training and Dev/Test Sets

5.1 Training and testing on different distributions

DL's data hunger often forces train and dev/test to differ (e.g. millions of web cat images + few thousand target app images).

- **Option 1 (bad):** shuffle everything, split randomly. All same distribution — but the target (app) distribution is now diluted in dev/test, so you optimize for the wrong thing.
- **Option 2 (good):** put *all* target-distribution data into dev/test (plus some in train); fill the rest of train with web data. Dev/test now reflect what you care about.

5.2 Bias/variance with mismatched distributions

When train and dev come from different distributions, a train-dev gap could be variance *or* the distribution difference. Disentangle with a **train-dev set**: same distribution as train, held out from training.

$$\begin{aligned}\text{Avoidable bias} &= \text{train err} - \text{human err}, \\ \text{Variance} &= \text{train-dev err} - \text{train err}, \\ \text{Data mismatch} &= \text{dev err} - \text{train-dev err}, \\ \text{Overfit to dev} &= \text{test err} - \text{dev err}.\end{aligned}$$

Example: human 0%, train 1%, train-dev 9% $\Rightarrow$ *variance* problem. But human 0%, train 1%, train-dev 1.5%, dev 10% $\Rightarrow$ *data mismatch* problem.

5.3 Addressing data mismatch

No fully systematic fix, but:

- Do **manual error analysis** to understand how train and dev/test differ.
- Make training data more like the target: collect more similar data, or use **artificial data synthesis** (e.g. add car noise to clean speech; render cars with 3D graphics).

Artificial synthesis can *overfit* to a tiny subset of the space — e.g. if you add the same 1 hour of car noise to 10,000 clips, the model may memorize that noise. Use diverse synthesis.

6. Week 2c — Learning from Multiple Tasks

6.1 Transfer learning

Reuse knowledge from task A (lots of data) on task B (little data). Delete the last layer of A's trained network, add new final layer(s), and:

- **Little data for B:** freeze the earlier layers, train only the new last layer(s).
- **More data for B:** fine-tune more (or all) layers. (Training A = *pretraining*; adjusting on B = *fine-tuning*.)

Makes sense when: A and B share the same input type (both images / both audio); you have *much* more data for A than B; low-level features from A are useful for B.

6.2 Multi-task learning

Train *one* network to do several tasks *at once* (one image, multiple labels — pedestrian, car, stop sign, light). Output $\hat{y}$ is a vector; loss sums over the components:

$$J = \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^c L(\hat{y}_j^{(i)}, y_j^{(i)}),$$

with logistic loss per component. Works even with *partially labeled* data (sum only over labeled j). **Makes sense when:** tasks share low-level features; each task has similar amounts of data; you can train one big-enough network. Sharing usually beats training separate networks. In practice transfer learning is used more often than multi-task (multi-task shines in e.g. object detection).

7. Week 2d — End-to-end Deep Learning

7.1 What it is

Replace a multi-stage pipeline with a *single* network mapping input directly to output.

Non-end-to-end	audio → features → phonemes → words → transcript
End-to-end	audio → transcript

End-to-end gives the data more freedom (may skip hand-designed steps like phonemes) — but needs a *lot* of data. With little data, the staged pipeline often wins (e.g. face recognition: detect face, then compare — each stage has more data than the whole; child age from X-ray: image→bones→age beats image→age).

7.2 Whether to use it

- **Pros:** lets the data speak (fewer human preconceptions); less hand-designing of components.
- **Cons:** needs large data; excludes potentially useful hand-designed components (which help more on small data).

Key question: do you have enough data to learn a function of the complexity needed to map $x \rightarrow y$ directly? If not, break the problem into pieces you *do* have data for.

Big picture of Course 3: this course is a decision framework, not algorithms. Set *one* metric on a dev/test set that matches your real goal → use human-level performance to split error into avoidable bias vs. variance → let error analysis and the train-dev set tell you *which* problem (bias / variance / data mismatch) to attack → and choose your setup (transfer, multi-task, staged vs. end-to-end) by how much data you actually have. Orthogonalization throughout: one knob, one effect.